

1 Problem Description

I build a navigation framework that moves a mobile robot from a start location to a goal location in a partially known 2D environment while avoiding obstacles and respecting the robot's motion constraints. The robot is a differential-drive platform: it cannot translate sideways and its linear and angular velocities are bounded and acceleration-limited, so a valid plan must be more than a sequence of free cells, it must be drivable.

The framework combines two complementary layers. A global planner searches the whole environment for a complete, collision-free route, providing long-term guidance. A local planner/optimiser then tracks that route while continuously scoring candidate motions against the robot's dynamics and nearby obstacles.

Why global planning is necessary, and why local planning alone is not. A local planner only looks a short distance ahead, so it always tries to move straight toward the goal. This works in open space but gets stuck when the goal is hidden behind an obstacle. The clearest example is a dead end: if the robot drives into a U-shaped wall, the goal is on the far side of it, so every “move closer” step just pushes the robot into the wall. Escaping would mean first moving *away* from the goal, which a local planner never chooses — so it stays trapped. A global planner avoids this by looking at the whole map at once and routing around such traps before the robot even starts. The two layers are complementary: the global planner decides which way to go, the local planner how to drive there smoothly.

2 System Architecture

The system is a modular pipeline; each stage is an independent Python module so components (e.g. the local optimiser) can be swapped without touching the rest.

1. **Map generation** (`utils/map_generator.py`) — produces a random binary occupancy grid with free start/goal cells.
2. **Probabilistic map** (`src/prob_map.py`) — converts the grid into a continuous Gaussian risk field and derives an inflated planning grid.
3. **Global planner** (`src/a_star.py`) — A* search on the inflated grid returns a waypoint path.
4. **Local planner** (`src/mppi.py`, primary; `src/dwa.py` as a baseline) — at each control step, evaluates candidate motions against a cost function and returns a velocity command.
5. **Navigation loop** (`src/navigation.py`) — drives the robot with a pure-pursuit look-ahead target along the path, integrating the motion model until the goal is reached.

6. **Visualisation** (`utils/visualisation.py`) — renders the grid, risk field, global path and executed trajectory.

MPPI is the operational planner; the `PLANNER` flag in `main.py` can switch to the DWA baseline for comparison. Both expose the same call signature, so the navigation loop is agnostic to which one runs.

3 Environment Representation

Binary occupancy grid. The world is an $N \times N$ grid G with $G(r, c) \in \{0, 1\}$ (0 = free, 1 = obstacle), generated randomly at a chosen obstacle density with guaranteed-free start and goal cells.

Probabilistic risk field A binary grid treats every cell as certainly free or certainly blocked, which ignores sensor uncertainty and lets a plan graze obstacle corners. I instead convolve G with a 2D Gaussian kernel K_σ to obtain a continuous risk field $\rho(r, c) \in [0, 1]$:

$$\rho = \text{clip}(\max(G * K_\sigma, G), 0, 1), \quad K_\sigma(r, c) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{r^2+c^2}{2\sigma^2}\right). \quad (1)$$

Risk decays smoothly with distance from obstacles ($\sigma = 0.8$ cells), so the local planner can reason about how safely a trajectory passes, not merely whether it collides. Continuous positions are queried by bilinear interpolation of ρ .

Inflated planning grid, and why representation matters. The representation a planner uses directly determines plan quality. The global and local layers must agree on what is traversable: if A^* searches the raw grid but the local planner rejects any cell with $\rho \geq \rho_{\text{thresh}}$, A^* returns corridors the robot considers lethal and it stalls. I therefore derive an inflated planning grid, $G_{\text{inf}} = [\rho \geq \rho_{\text{thresh}}]$, and run A^* on it, so the global route only visits cells the local planner also deems safe. This single alignment choice is what makes the two layers cooperate rather than fight.

4 Global Planning Method: A^*

I use A^* for global planning: like Dijkstra it is complete and (with an admissible heuristic) optimal, but its goal-directed heuristic expands far fewer nodes, making it the natural choice on an occupancy grid.

Each node n is scored by

$$f(n) = g(n) + h(n), \quad (2)$$

where $g(n)$ is the actual cost from the start and $h(n)$ is the estimated cost to the goal. I use the Euclidean distance as h , which is admissible because it never overestimates the true remaining cost, $h(n) \leq h^*(n)$ — so A^* returns a shortest path. The search expands the open-list node of lowest f , relaxing 8-connected neighbours; diagonal moves are allowed only when both flanking orthogonal cells are free, preventing paths that cut through obstacle corners.

A^* runs on the inflated planning grid (Section 3), so the resulting waypoint list is a complete, collision-free, and, what is more important, drivable reference for the local planner. Because the search covers the whole map, it provides the global guidance that lets the system escape the dead-end traps a local method cannot.

5 Local Optimization Method: MPPI

My local planner is a **Model Predictive Path Integral** controller. It is a sampling-based stochastic optimal-control method: at every control step it evaluates many candidate motions against a cost function and returns the best velocity command, continuously adapting to the nearby obstacle geometry.

Motion model. The robot state is $(r, c, \theta, v, \omega)$ and evolves under a differential-drive model integrated at Δt :

$$\theta += \omega \Delta t, \quad r += v \cos \theta \Delta t, \quad c += v \sin \theta \Delta t. \quad (3)$$

The robot cannot translate sideways and its velocities are bounded, so these constraints are respected directly in the rollout.

Sampling-based optimisation. Rather than committing to a single constant command, MPPI optimises a whole control sequence $U = (u_0, \dots, u_{T-1})$ over a short horizon T . Each step it draws K noisy rollouts $U^{(k)} = U + \mathcal{E}^{(k)}$ (clipped to the velocity limits), simulates each with the motion model, scores each trajectory cost $S(\tau^{(k)})$, and updates the nominal sequence by a softmin-weighted average of the perturbations:

$$w_k = \frac{\exp(-\frac{1}{\lambda}(S_k - \min_j S_j))}{\sum_j \exp(-\frac{1}{\lambda}(S_j - \min_i S_i))}, \quad U \leftarrow U + \sum_k w_k \mathcal{E}^{(k)}. \quad (4)$$

It then executes the first command u_0 and shifts U forward one step (receding horizon), which warm-starts the next optimisation and yields smooth, temporally consistent commands. Because it samples full sequences and softly averages over many rollouts, MPPI can represent compound manoeuvres (“turn, then go straight”) and steer around local minima.

Objective function. Each rollout is scored by a weighted sum (lower is better):

$$S(\tau) = w_g c_{\text{goal}} + \alpha c_{\text{head}} + \beta c_{\text{obs}} + \gamma c_{\text{vel}} + \delta c_{\text{smooth}} + \epsilon c_{\text{dev}}, \quad (5)$$

whose terms reward closing distance to the goal, heading alignment, obstacle clearance, forward speed, and low angular jerk and staying near the global path. The goal term is the primary driver of progress; heading alone only aligns orientation and gives no incentive to actually advance.

Weighting, and speed/safety/efficiency trade-offs. The weights encode the robot’s “personality” and expose direct trade-offs. A high β (obstacle) yields wide, cautious detours; a high γ or w_g yields fast but riskier passes close to obstacles; a high ϵ keeps the robot glued to the global path at the cost of local shortcuts. No single weighting is best — the right balance of safety, speed and efficiency depends on obstacle density — and because MPPI re-solves this optimisation every control step against the current geometry, it reacts to obstacles rather than blindly replaying the global path.

6 Optional Extensions

I implemented two of the optional extensions, both of which feed directly into the MPPI cost function.

Probabilistic Mapping. The Gaussian risk field replaces the binary obstacle test with a continuous one. In the planner the discrete “infinite cost inside the safety radius” rule becomes a smooth penalty on the risk along a trajectory, with a hard cutoff only at $\rho \geq \rho_{\text{thresh}} = 0.5$. This gives the optimiser a gradient that steers it away from obstacles *before* a collision is imminent. Concretely, the obstacle term averages a soft risk penalty over the horizon and adds a large finite penalty for any sample that breaches the threshold, so guidance never freezes the robot yet true collisions are ruled out.

Multi-Objective Optimization. I add two objectives to the cost: a *smoothness* term $c_{\text{smooth}} = \frac{1}{T} \sum_t |\omega_t|$ that penalises jerky turning, and a *path-deviation* term $c_{\text{dev}} = \min_i \|\tau_T - p_i\|$ that keeps the trajectory endpoint near the global path $\mathcal{P}$. The planner therefore optimises several criteria at once — progress, safety, smoothness and path consistency — which is what exposes the trade-offs discussed in Section 5. Both terms are tunable independently through the planner configuration, so their relative influence can be studied without changing the algorithm.

7 Experimental Results

Setup. To evaluate the local planner I ran a batch of seeded random 20×20 maps at 30% obstacle density, restricted to non-trivial start-goal pairs (global path ≥ 8 waypoints), each with a 600-step budget.

Comparison against a DWA baseline. Before settling on MPPI I also implemented the classic **Dynamic Window Approach** and ran it through the identical pipeline as a baseline. DWA evaluates only a single constant command held over the horizon, so every candidate motion is a circular arc, and from a standstill its acceleration-limited search window admits only near-zero speeds. Table 1 reports both planners on the same maps.

Local planner	Reached goal	Avg. steps (successful)
MPPI (chosen method)	17 / 25 (68%)	208
DWA (baseline)	0 / 25 (0%)	—

Table 1: Goal-reaching success over 25 non-trivial maps. Both planners use the same aligned A* path and probabilistic cost; only the optimiser differs.

MPPI reached the goal on 68% of maps while DWA, under the identical pipeline, reached none — it stalls in the local-minima traps analysed in Section 8, which is exactly why I chose MPPI as the primary method. Figure 1 shows a representative MPPI run: the executed trajectory (red) tracks the A* path (dashed blue) through the clutter while staying in the low-risk (green) band of the field, rounding corners smoothly rather than cutting them.

I also observed the effect of weighting: raising the obstacle weight widens the robot’s clearance but slows it and can freeze it in tight passages, while raising the velocity/goal weights speeds it up at the cost of closer, riskier passes — a concrete speed/safety trade-off.

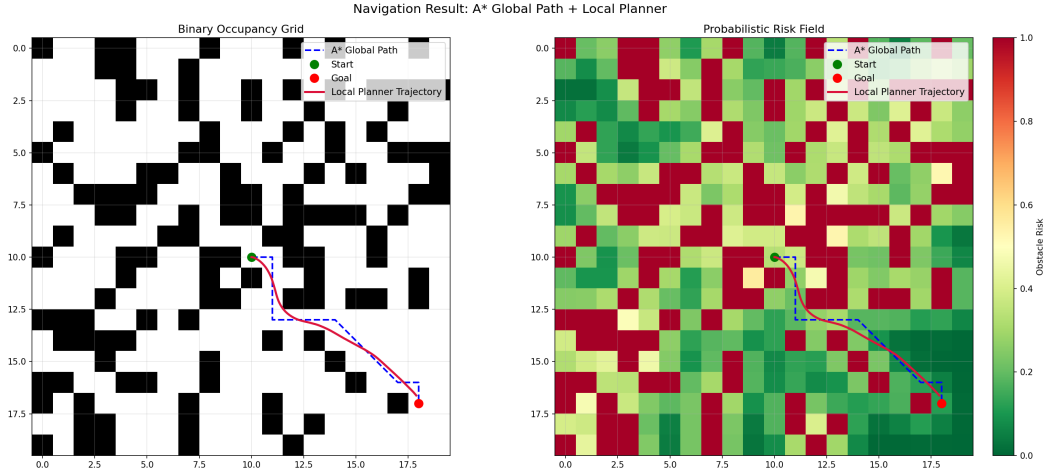


Figure 1: Navigation on a 20×20 map (30% density). Left: binary grid. Right: probabilistic risk field (red = high, green = safe). Dashed blue: A* global path. Solid red: executed MPPI trajectory.

8 Discussion and Failure Cases

Building the system surfaced several instructive failure modes, most of them in the local layer.

DWA rotate-in-place / frozen robot. DWA fails structurally in clutter (0/25). From a standstill the acceleration-limited window admits only near-zero speeds, so a forward candidate barely displaces the robot over the horizon; the progress terms cannot distinguish candidates while the obstacle term can, and pivoting in place wins. The robot never builds speed, so the window never opens resulting into a frozen-robot local minimum. A related earlier bug amplified this: querying the risk field by nearest-cell rounding created cost discontinuities at cell boundaries; replacing it with bilinear interpolation removed those artefacts. DWA also rejects any rollout that clips an inflated cell as infinite cost, which near corners can eliminate every forward option.

Why MPPI helps, and where it still fails. MPPI avoids these traps by optimising full control sequences and softly averaging over rollouts, reaching 68% of goals. Its remaining 32% of failures occur in the tightest passages, where the sampling distribution rarely proposes the narrow feasible manoeuvre within the horizon; a longer horizon or more samples trades compute for a higher success rate. In these 32% there are some paths which are simply cannot be executed because of the map layout.

Trade-offs. Across both planners the recurring tension is speed vs. safety vs. efficiency: cautious weightings are safe but slow and freeze-prone, aggressive weightings are fast but graze obstacles. The probabilistic field mitigates this by turning a hard constraint into a smooth gradient, but the weights still require fine-tuning.

Limitations and future work. The environment is static and fully observed, so dynamic obstacles and true partial observability are not yet handled, and DWA still needs manual weight tuning and stays freeze-prone. Natural next steps: online replanning when the map changes, a

sampling-based global planner (RRT*) for continuous spaces, learning the cost weights instead of hand-tuning, and an explicit recovery behaviour for the frozen-robot state.

9 Reflection: What Makes the System Intelligent?

The system’s intelligence is not in any one algorithm but in the interaction of all ingredients.

Search, optimisation, and representation together. **Search** (A*) supplies global competence — completeness and optimality over the map, so the robot never traps itself in a dead end. **Optimisation** (DWA/MPPI) supplies local competence — continuously choosing dynamically feasible motions that trade off progress, safety and smoothness against the live obstacle geometry. **Representation** (the probabilistic risk field and the inflated planning grid) lets the two cooperate: it turns a binary world into a graded one and guarantees both layers agree on what is drivable. Remove any one and the whole degrades: no search gives dead ends, no optimisation undrivable paths, no shared representation two layers that fight.

Reasoning about the future. Intelligence here is not a reflex to the current instant. MPPI optimises a whole control sequence over a short horizon, scoring where each candidate *leads* rather than how it looks now — so the robot commits to compound manoeuvres (“turn, then go straight”) and accepts a momentarily worse state to reach a better one, which a greedy step cannot do.

Balancing competing objectives. The robot never optimises a single quantity; every motion is judged by a weighted trade-off of progress, clearance, speed, smoothness and path fidelity. Behaving well means *negotiating* these goals when they conflict — closer to judgement than to a fixed rule.

Adaptation and generality. The system is reactive: it re-solves its cost minimisation every control step, with MPPI’s receding-horizon warm start carrying information forward, so the robot answers the obstacles in front of it. Nothing is hand-crafted for a specific map, so the same pipeline solves arbitrary random worlds — competence in the method, not a memorised solution.